Práctica 4

Sesiones 9, 10 y 11

Objetivos

- Conocer los fundamentos del lenguaje de programación JavaScript.
- Entender la programación web en el cliente.
- Comprender cómo JavaScript sirve para crear webs dinámicas.
- Ser capaz de desarrollar guiones en JavaScript utilizando todas las prestaciones que ofrece el lenguaje.

Introducción a JavaScript

Conforme la Web gana popularidad, se requiere cada vez más un lenguaje de programación que se ejecute en el cliente. Brendan Eich creó un lenguaje denominado LiveScript a mediados de los 90 del siglo pasado, que llegó a denominarse JavaScript más tarde. En 1997 fue estandarizada la versión 1.1 de este lenguaje, aunque realmente se diseñó un estándar denominado ECMAScript el cual es la base para las implementaciones de JavaScript (JavaScript es el lenguaje de programación, y se puede considerar una implementación del estándar ECMAScript). Actualmente está en vigor la versión 9 de ECMAScript (2018). Éste incluye dos partes más: Document Object model (DOM) y Browser Object Model (BOM). Javascript es un lenguaje interpretado que se ejecuta en el navegador.

Fundamentos de JavaScript

He aquí algunas de las consideraciones básicas del lenguaje de programación:

- El código JavaScript debe ir en un elemento script de HTML:

```
<script type="text/javascript">
        alert("Hola, Mundo");
</script>
```

Ya no es necesario indicar el tipo del guión, pues JavaScript es el lenguaje por defecto.

- También el código JavaScript puede ir en un fichero externo. Para ejecutarlo se emplea el atributo *src* de *script*:

```
<script  src="miGuion.js"></script>
```

- Los guiones de JavaScript pueden ir indistintamente en los elementos *head* o *body*. Lo habitual es colocar las funciones en el primero, o al final de la página. Colocándolo en el *head*, se está indicando que todo el código JavaScript debe ser descargado, analizado e interpretado antes de

que la página comience a mostrarse. Esto puede suponer un tiempo adicional de descarga largo, razón por lo cual se pueden meter en el *body*, en cuyo caso los scripts se cargarán y ejecutarán cuando los elementos anteriores (más arriba en el código) de la página ya se hayan mostrado. Se ejecutan cuando se termine de analizar la página si tienen el atributo *defer*. Si no, justo cuando se encuentre.

- Los comentarios se hacen mediante /* … */ ó //.
- Operadores aritméticos: +,-,*, / y %. Precedencia igual que en otros lenguajes. Se emplean los paréntesis para establecer un orden de evaluación requerido.
- Cadenas de caracteres entre comillas dobles o simples (*"Hola. Esto es una cadena de caracteres." y 'esto, también'*).
- El operador de concatenación de cadenas de caracteres es + (*"hola"* + *"adiós"*).
- Las variables booleanas almacenan *true* y *false*.
- El carácter '\' se emplea como indicador de caracteres con significado.
- '*n*' y '*t*' para introducir una nueva línea y un tabulador, respectivamente.
- El operador *typeof* devuelve una cadena con el tipo de dato del operando (*typeof 4.5* devolvería "*number*". Se puede utilizar también como una función let *resultado=typeof("hola"); (*devuelve *"string"*).
- Los valores numéricos se pueden comparar mediante los operadores relacionales '>=', '<=', '<', '>', '!=' y '==''. Estos operadores son de aplicación a las cadenas de caracteres y devuelven los valores *true* o *false*.
- Los operadores lógicos son: '&&', '| |' y '!', y devuelven los valores *true* o *false*.
- false será equivalente a una cadena vacía (" "), 0 ó *NaN*, *null* y *undefined*. *null* y *undefined* se también se emplean para indicar que las variables están vacías. La primera, es un valor especial que indica nada, vacío o valor desconocido; la segunda, que el valor no ha sido asignado.
- Todas las sentencias de JavaScript finalizan con un ';' o no… cuando hay una nueva línea para finalizar la sentencia se puede dejar sin ;.
- Para declarar variables se emplea la palabra reservada *var* (var altura=5.5;). Las variables se pueden nombrar incluyendo los caracteres '$' y '_'. No se puede emplear ninguna variable que no haya sido declarada previamente, aunque sea en el momento de asignarle un valor.
- También se puede usar *let* para hacer la asignación (let altura=5.5;). La diferencia entre let y var es la siguiente: la primera limita el alcance de la variable declarada al bloque de ejecución, expresión o declaración en la que se encuentre; la segunda, al ámbito de la función o al global (si no se encuentra en ningún bloque). Actualmente se emplea *let* en lugar de var, que se considera anticuado y ya está en desuso.

```
{
        let variable_let = 'valor variable_let';
        var variable_var = 'valor variable_var';
         console.log('var en el bloque {}: ' + variable_var);
         console.log('let en el bloque {}: ' + variable_let);
}
console.log('var fuera de bloque: ' + variable_var);
console.log('let fuera de bloque: ' + variable_let);
```

```
let var1 = 5;
if (var1 > 1) {
                let var2 = 10;
                var var3 = 15;
                console.log(var1 + var2 + var3); // Se escribe 30
        }
console.log(var1 + var2 + var3);  // Daría error pues var2 no existe aquí.
console.log(var1 + var3); // Se escribe 20.
```

- Se puede declarar una variable pero no asignarle ningún valor inicialmente. En este caso, el tipo de la misma es *undefined: let mensaje; resultado=typeof(mensaje); // resultado toma undefined.*
- Para declarar una constante se usa la palabra reservada *const*.
- Cuando se empieza a ejecutar un programa JavaScript, se crea un entorno de variables, junto con sus valores correspondientes. Este entorno está inicializado con un conjunto de variables estándares que el navegador cargará y asignará valores. Este entorno se comparte entre distintos bloques *<script>* de la misma página (al igual que se comparte en PHP entre distintos bloques *<?php ?>*).
- Las funciones en JavaScript se invocan igual que en otros lenguajes. Pueden o no devolver un valor.
- Ventanas de mensajes o diálogos incluidos en el entorno estándar:
  - *alert("Texto");* → muestra una ventana con el texto en su interior. También puede mostrar números.
  - *confirm("Iremos?");* → Muestra el texto así como dos botones (OK/Cancel) y devuelve *true* o *false* según se haya hecho click en OK o en Cancel, respectivamente.
  - *prompt("Dime tu película favorita:", "...");* → Muestra el texto pasado como primer argumento y un campo de texto para meter un texto de entrada. El segundo argumento es el texto que aparecerá por defecto en el campo de texto. Devuelve el texto introducido o *null* en caso de que el usuario haya pulsado el botón de cancelar.
- Para escribir texto en la propia página web podemos usar la función *document.write('texto'),* o si simplemente queremos tener un control sobre las salidas de ciertas funciones podemos usar la consola de javascript mediante la función *console.log('texto').* Para poder ver dichos valores tendremos que acceder a dicha consola (en el navegador Chrome: menú->herramientas->Consola de JavaScript). Actualmente está en desuso y se suele emplear la consola para depurar y manipulación del DOM para mostrar resultados.
- Operador "Nullish Coalescing" (??): devuelve el lado derecho si el izquierdo es null o undefined:

```
let cantidad = 0;
let resultado = cantidad ?? 100;   // Devuelve 0 (porque 0 no es null/undefined)


let nombre = null;
let mostrar = nombre ?? "Anónimo";  // Devuelve Antónimo "Anónimo"
```

- Las estructuras repetitivas de JavaScript son los bucles *while* y *for*:

```
let contador = 0;
while (contador <= 12)
        contador = contador + 2;
for (let contador = 0; contador <= 12; contador = contador + 2)
        document.write(contador);
```

Para los cuerpos de bucles de más de una línea, se emplean las llaves para delimitar su ámbito. Se emplea la sentencia *break* para terminar la ejecución del bucle y *continue* para saltarse la iteración actual y pasar a la siguiente.

- Existen los operadores ++, - -, +=,...
- Las estructuras condicionales de JavaScript son los *if* y *switch*:

```
for (let cont = 0; cont < 20; cont++) {
                if (cont > 15)
                        document.write(cont + "**");
                else if (cont > 10)
                        document.write(cont + "*");
                else  document.write(cont);
}
let dia=new Date().getDay();

switch (dia)
{
case 0: x="Domingo";
        break;
case 1: x="Lunes";
        break;
case 2: x="Martes";
        break;
case 3: x="Miércoles";
        break;
case 4:x="Jueves";
        break;
case 5:x="Viernes";
        break;
case 6:x="Sábado";
        break;
}
```

Funciones

- Las funciones se declaran mediante la palabra reservada *function*.

- Pueden devolver o no valores. En caso de hacerlo, se emplea la palabra return (sin que sea necesario usar paréntesis), y si no, realmente devuelven un valor "undefined".

```
function add(a, b) {
        return a + b;
}
 document.write(add(2, 2));
function mostrarMensaje(mensaje) {
  alert("¡¡"+ mensaje + "!!");
}
mostrarMensaje("OOOOleeeeee");
```

- Los nombres de funciones pueden asignarse a variables y luego emplearlas:

```
function mostrarMensaje(mensaje) {
        let funcion=alert;
          funcion("¡¡"+ mensaje + "!!");
}
mostrarMensaje("OOOOleeeeee");
```

- Las variables declaradas dentro de una función son locales a la función, pero las que se declaran fuera pueden ser empleadas dentro de las funciones.
- Los parámetros se pasan por copia, excepto los arrays y objetos, que se pasan por referencia.
- Se pueden declarar parámetros con valores por defecto:

```
function showMessage(from, text = “defecto”) {...}
```

- Se pueden declarar funciones dentro de otras, en cuyo caso, el entorno local de las primeras contendrá el de las segundas:

```
let variable = "fuera";
function funcionPadre() {
         let variable = "local";
        function funcionHija() {
                document.write(variable);
         }
        funcionHija();
}
funcionPadre();
```

- OJO: sólo se crean entornos locales a partir del cuerpo de las funciones.
- *Se crean entornos de variables y las variables declaradas con let no están disponibles fuera.*
- *Por ejemplo: {let a = 2;} document.write(a); / / a no está definida aquí.*
- Se pueden devolver funciones internas y luego invocarlas:

```
let variable = "fuera";
```

```
function funcionPadre() {
            let variable = "local";
             function funcionHija() {
                document.write(variable);
             }
            return funcionHija;
           }

let hija = funcionPadre();
hija();

function funcionParaSumar(cantidad) {
           function sumar(numero) {
                                return numero + cantidad;
            }
 return sumar;
}
let aniadirDos = funcionParaSumar(2);
let aniadirCinco = funcionParaSumar(5);
 document.write(aniadirDos(1) + aniadirCinco(1));
```

- No sólo se devuelve el nombre de la función, sino que también implícitamente se devuelve el entorno asociado.
- Se permite recursividad.
- Existe el array *arguments* que almacena todos los argumentos pasados a una función. Sólo se puede emplear dentro de ella (el primer argumento está en la posición 0). Con *arguments.length* podremos saber cuántos se han pasado a la función. Esto es útil ya que en Javascript, las funciones pueden llamarse con un mayor número de parámetros de los que se definieron en la declaración (ej.: *function a(){…} a(1,2,3,4);*).
- Actualmente se emplean los "Rest Parameters" en su lugar, que empaquetan todos los argumentos que no tienen un nombre específico y se representan por "..." seguido del nombre del Array que recogerá el resto no nombrados. Sólo puede haber un parámetro de este tipo y tiene que ir al final de la lista de parámetros en la declaración de la función:

```
function miLista(primero, segundo, ...elResto) {
    console.log("Primero:", primero);
    console.log("Segundo:", segundo);
                console.log("El resto:", elResto); // Esto es un Array real
}

miLista("Manzana", "Pera", "Uva", "Sandía", "Mango");
```

```
// Resultado:
// Primero: Manzana
// Segundo: Pera
// El resto: ["Uva", "Sandía", "Mango"]
```

- No se pueden sobrecargar funciones (por ejemplo, no podemos definir a la vez la función *foo(x)* y *foo(x,y,z)*, aunque ambas tengan un distinto número de parámetros).
- Se pueden emplear parámetros por defecto, al estilo de lenguajes como PHP.
- Se pueden crear valores función al estilo a como se crean e inicializan variables:

```
let suma = function(a, b) {
        return a + b;
};  /* OJO con este ; pues es como si hiciéramos suma=3; */
 document.write(suma(5, 5));




function funcionParaSumar(cantidad) {
        return function (numero) {
                return cantidad + numero;
        };
}
let suma= funcionParaSumar(5);
 document.write(suma(3));
```

- Operador *arrow* "=>", utilizado para declarar funciones de manera sencilla:

```
 let suma = (a, b) => a + b;
/* O lo que es lo mismo: */
let suma = function(a, b) {
 return a + b;
};
```

- Manejo de errores con try/catch:

```
try {
  let a = 5/0;


 } catch (err) {
  console.error("Error capturado:", err.message);
      }
}
```

Objetos

- En JavaScript todo son objetos (*Object*, de forma genérica; una cadena de caracteres, *String*; un número, *Number*; un vector, *Array*; una función, *Function*).
- Los objetos son tipos de datos especiales que tienen propiedades y métodos.
- En JavaScript no existe el concepto de clase. No se crean clases explícitamente.
- Las propiedades se acceden mediante un punto, *cadena.length*, o alternativamente mediante la sintaxis de arrays, *cadena["length"];*, donde el nombre de la propiedad (*length* en este caso) debe ir entre comillas, al ser un string.
- Los métodos, con la sintaxis clásica: *cadena.toUpperCase();*
- La creación de objetos se puede hacer de dos formas diferentes:

```
libro= new Object();
libro.autor="Cervantes";
libro.titulo="Don Quijote de la Mancha";
libro.anio=1615;
libro.isbn="343.4335.234";
/* ó */
libro={autor:"Cervantes", titulo:"Don Quijote de la Mancha", anio:1615, isbn:"343.4335.234"};
```

- Los constructores son funciones que se denominan igual que el objeto. Se llaman anteponiendo *new* al nombre del constructor, el cual devolverá el objeto creado:

```
function Libro(autor,titulo,anio,isbn)
{
        this.autor=autor;
        this.titulo=titulo;
        this.anio=anio;
        this.isbn=isbn;
}
let quijote= new Libro("Cervantes", "Don Quijote de la Mañcha", 1615, " 343.4335.234");
```

- Se pueden añadir propiedades en cualquier momento. Simplemente asignándole un valor:

```
quijote.editorial="Salvat";
```

- Se puede emplear *this* para acceder a un dato miembro de un objeto.
- Los métodos se asignan a un objeto añadiéndolos dentro del constructor correspondiente:

```
function Libro(autor,titulo,anio,isbn)
{
        this.autor=autor;
        this.titulo=titulo;
        this.anio=anio;
        this.isbn=isbn;
```

```
}
```

- Se pueden crear propiedades que no tengan un nombre como tal, sino, por ejemplo, un objeto cadena de caracteres:

```
let mensaje={"De": "Juanma", "Para": "Jose", "Asunto":"Asignatura", "Cuerpo":"Hola, Jose...."};
```

- Para acceder a los valores que almacena, el objeto se indexa como un vector por dicha propiedad:

```
mensaje["De"]="Pepe";
```

- Existe un operador para comprobar si una propiedad existe en un objeto. Este es *in*, el cual devuelve *true* si está y *false* en caso contrario:

```
let resultado= "De" in mensaje;
 document.write(resultado);
```

- Para borrar una propiedad se emplea *delete*: *delete mensaje["De"];*
- Se puede comprobar si existe una propiedad comparándola con *undefined*:

```
let resultado= {}
let existe = (resultado.propiedad === undefined);
// Devuelve falso. === compara tanto el valor como el tipo de los operandos
```

- Para obtener los valores de los atributos miembro de un objeto se emplea el siguiente bucle:

```
let profesor={nombre:"Juanma",apellidos:"Fernández"};
for (x in profesor)
        { datos_profesor=datos_profesor + profesor[x]; }
```

- Cuando disponemos de una referencia a un objeto, se puede indicar que no apunte a ningún objeto asignándole *null*.
- Para conocer si un objeto es instancia de un constructor se emplea: let *res= quijote instanceof libro;* Otro ejemplo: let *res2= mivector instanceof Array;*
- Los métodos son propiedades que almacenan una función. Podemos añadir un método fuera del constructor asignándoselo a un objeto concreto o asignándoselo al constructor, de la siguiente forma:

```
Libro.prototype.mostrarAutor = function() {
  alert(this.autor);
}
```

- Los datos miembro pueden ser *configurable*, si puede borrarse mediante *delete*; *enumerable*, si puede ser devuelta por un bucle for-in; *writable*, si puede ser cambiada, y *value*, que contiene el valor real de la propiedad. Por defecto, al añadir una propiedad a un objeto, ésta es configurable, enumerable y writable. Para cambiar y asignar una de estas propiedades, se emplea el método *defineProperty*:

```
let persona = {};
Object.defineProperty(persona, "nombre", {
        writable: false,
        configurable: false,
        value: "Juanma"
```

```
});
```

- Una vez que una propiedad se ha asignado como no configurable, no se puede cambiar. Cualquier intento, da lugar a una excepción.
- Existen también "Accessor properties" (*configurable, enumerable, get* y *set*). Las dos últimas son funciones para obtener y devolver el valor de una propiedad.

```
let libro = {
	_anio: 2004,
	edicion: 1
};
// Estamos definiendo una nueva propiedad del objeto libro llamada anio, la cual tiene un getter y un setter.
// Si queremos definir la propiedad anio para cualquier objeto obtenido llamando a un constructor con new,
	// entonces hay que sustituir libro por NombreConstructor.prototype
Object.defineProperty(libro, "anio", {
	get: function(){
		return this._anio;
	},
	set: function(valor){
		if (valor > 2004) {
			this._anio = valor;
			this.edicion += valor - 2004;
		}
	}
});
libro.anio = 2005; // OJO, no le estamos asignando directamente un valor a _anio.
alert(libro.edicion); //2
```

- Cuando se precede el nombre de una propiedad mediante "_" se está dando a entender que no será accedida directamente, sino implícitamente mediante *get* y *set*.
- Para definir varias propiedades a la vez:

```
let libro = {};
Object.defineProperties(libro, {
		_anio: {
			value: 2004
			},
		edicion: {
			value: 1
			},
		anio: {
```

```
                    get: function(){
                    return this._anio;
                    },
                    set: function(valor){
                        if (valor > 2004) {
                        this._anio = valor;
                        this.edicion += valor - 2004;
                        }
                    }
                }
});
```

- Operador "Optional Chaining": si al acceder a una propiedad que era null o undefined, JavaScript daba un error y detenía toda la ejecución de un programa. Con el operador "?.", se verifica la parte izquierda del operador y si la referencia es null o undefined, se devuelve undefined y si la referencia existe, se continúa hacia la siguiente propiedad.

```
const usuario = {
            id: 1,
        info: {nombre: "Alex"}
};

// Acceso seguro a una propiedad existente
console.log(usuario.info?.nombre); // Resultado: "Alex"

// Acceso seguro a una propiedad que NO existe
console.log(usuario.contacto?.telefono); // Resultado: undefined (¡No hay error!)

// Sin el chaining, esto lanzaría un error:
// console.log(usuario.contacto.telefono); // Uncaught TypeError...
```

- Prototipos y herencia: En JavaScript los objetos tienen una propiedad oculta e interna denominada ~~property~~ *no te refieres a prototipo o __proto__?*. Cuando queremos acceder a una propiedad de un objeto que no existe, la toma de la propiedad ~~property~~ __proto__.

```
    let animal = {
        come: true,
        anda()
        {alert("Animal anda");}
        }; //anda(){...} equivalente a anda: function(){}
    let conejo = {
```

```
            salta: true;
            };
conejo.__proto__ = animal; // Hacemos que conejo herede de animal

alert(conejo.come);
alert(conejo.salta);
// ----------------------
  let animal= {
            come: true
            anda()
            {alert("Animal anda");}
            };
let conejo = {
            salta: true;
            __proto__: animal;
            };
conejo.anda();
// --------------------------
  let animal= {
            come: true
            anda() {};
            };
let conejo = {
            salta: true;
            __proto__: animal;
            };
conejo.anda= function() {
alert("El conejito camina para adelante, el conejito camina para atrás");
```

Clases (A partir de ES6)

A partir de la versión 6 de ECMAScript, se incluye el concepto de clase en JavaScript, facilitando mucho más todo la programación orientada a objetos de JavaScript.

- La creación de clases se hace de la siguiente manera:

```
class Usuario {
        // Se definen los atributos sin el let
        let nombre =""
        let usuario;
```

```
        /* Constructor */
         constructor(nombre) {
                this.nombre = nombre;
            }
        /* Métodos */
            mostrarNombre() {
                alert(this.nombre);
                }
}
```

- Y su uso:

```
let usuario = new Usuario("Juanma");
usuario.mostrarUsuario();
```

- En JavaScript una clase es una función que toma el código del constructor. Los métodos se almacenan en el prototipo.
- Se pueden crear getter y setter en la clase:

```
 class Usuario {
_id = “1234”; // No se comparte por los objetos creados a partir de la clase.
_nombre =””;
 constructor(nombre) {
        this._nombre = nombre;
            }
        get nombre() {
            return this._nombre;
         }
        set nombre(valor) {
            if (valor.length < 4) {
                alert("Nombre demasiado corto”);
             return;
              }
            this._nombre = value;
 }
}
let usuario = new Usuario("Juanma");
alert(usuario.nombre);
usuario = new Usuario("");
```

En el enlace https://developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Classes podéis encontrar

un tutorial sobre clases, objetos y herencia en ES8.

Arrays

(http://www.w3schools.com/js/js_obj_array.asp)

- Los arrays se inician mediante corchetes: let *animales= ["perro", "gato", "león"].*
- Se asignan valores mediante un índice: *animales[4]="tigre"*. El acceso es el típico: *animales[1];*
- El primer índice del vector es 0.
- Se pueden crear arrays vacíos: ~~let~~ *let animales = [];*
- Realmente un vector es un objeto de la clase *Array*, por lo que se pueden aplicar métodos a dichos objetos:
  - Construcción: let *coches= new Array(); ó let coches= new Array("Seat", "Audi", "Volvo");*
  - Para incluir nuevos elementos se emplea el método *push*: *animales.push("elefante");* Se añade al final de array.
  - El número de elementos del array se obtiene mediante la propiedad *length* (*animales.length*).
  - El índice de un valor se obtiene por medio de *indexOf*: let *pos= coches.indexOf("Audi");* Si el array no contiene el valor, devuelve -1.
  - Para concatenar Arrays se emplea el método *concat* (*array1.concat(array2);*) o el operador "+"
  - "Template Literals":

```
    let usuario = "María";
let saludo = `Hola, ${usuario}. ¿Cómo estás?`;
console.log(saludo);   // Hola, María. ¿Cómo estás?

// También permiten saltos de línea
let carta = `
  Querido ${usuario}:
  Te escribo esta carta…
`;
```

  - Para obtener el último elemento se usa el método *pop*.
  - Otros métodos: *join, reverse, shift, slice, sort, splice, toString, unshift,...*
- Se pueden almacenar diferentes tipos de datos en un array, ya que todos los componentes de un Array se consideran objetos.
- Se pueden añadir elementos al vector en cualquier posición, aunque no esté lleno hasta ella.
- El bucle foreach sirve para iterar por un array (cuando se desea realizar una acción por cada elemento) - Ojo: no se pueden emplear ni break ni continue.

```
const frutas = ["manzana", "pera", "uva"];
// Recorrer y mostrar cada elemento:
```

```
frutas.forEach(fruta => {
  console.log(fruta);
});

//Uso del índice:
colores.forEach((fruta, i) => {
  console.log(`${i}: ${fruta}`);
});

// Modificación de elementos externos
let suma = 0;
const numeros = [2, 4, 6];
numeros.forEach(n => { suma += n; });
console.log(suma); // 12
```

- Algunos métodos modernos para transformación de arrays:

```
const numeros = [1, 2, 3, 4, 5];
// map -> crea un array aplicando una función a cada elemento
const dobles = numeros.map(n => n * 2); // [2,4,6,8,10] // Duplicación
// filter -> crea un nuevo array con los elementos que cumplen una condición
const pares = numeros.filter(n => n % 2 === 0); // [2,4] // Filtrar pares
// reduce -> reduce el array a un sólo valor acumulado
const suma = numeros.reduce((acum, n) => acum + n, 0); // 15 // Suma total
```

- Algunos de búsqueda:

```
const edades = [18, 22, 15, 30];
// find() : devuelve el primer elemento que cumple la condición.
// findIndex() : devuelve su índice.
const mayor18 = edades.find(edad => edad >= 18); // 18
// some() : devuelve true si al menos uno cumple.
const hayMenor = edades.some(edad => edad < 18); // true
every() : devuelve true si todos cumplen.
const todosMayores = edades.every(edad => edad >= 18); // false
```

- Spread operator (...) para arrays: permiten expandir o concatenar arrays de forma más clara:

```
const a = [1, 2];
```

```
const b = [3, 4];
const combinado = [...a, ...b];   // [1,2,3,4]
// Copia superficial
const copia = [...frutas];
// Útil para pasar elementos como argumentos
const maximo = Math.max(...numeros); // 5
```

Cadenas de caracteres

(http://www.w3schools.com/jsref/jsref_obj_string.asp.)

- Son objetos.
- Creación: let *nombre="Juan Manuel"; ó let nombre='Juan';* No hay diferencia entre "X" y 'X'.
- Se puede acceder a una posición concreta (los índices comienzan en 0): let *car=nombre[3];*
- Se emplea el carácter '\' para escapar otros caracteres.
- Longitud de la cadena: *nombre.length;*
- Encontrar la posición donde comienza una subcadena: *let pos=nombre.indexOf("Manuel");* Devuelve -1 si no está la subcadena.
- Sustitución de subcadenas: let *cad="Juan Manuel"; pos= cad.replace("Juan", "José");*
- Conversión a mayúsculas y minúsculas: *cad2=cad.toUpperCase(); y cad3=cad.toLowerCase();*
- Conversión de una cadena a un vector: *let str="a,b,c,d,e,f"; let n=str.split(",");* (Separador).
- Más métodos en http://www.w3schools.com/js/js_obj_string.asp.

Otros objetos

- Date (https://www.w3schools.com/js/js_dates.asp).
- Math (https://www.w3schools.com/js/js_math.asp).
- RegExp (https://www.w3schools.com/js/js_regexp.asp).

Exportaciones e importaciones

La exportación e importación de módulos es lo que permite dividir tu código en varios archivos, manteniendo todo organizado y evitando que el "archivo global" se convierta en un caos. En JavaScript moderno, se usa el estándar ESM (ECMAScript Modules). Existen dos formas de hacerlo:

1. Exportaciones nombradas: se usan cuando un mismo archivo tiene varias cosas que se desea compartir (funciones, variables, clases):
    - Exportar: se emplea la palabra export delante de cada elemento.
    - Importar: se debe usar llaves { } y el nombre exacto de lo que se exporta.

```
// archivo: matematicas.js
export const sumar = (a, b) => a + b;
```

```
export const PI = 3.1416;


// archivo: app.js
import { sumar, PI } from './matematicas.js';
console.log(sumar(2, PI));
```

2. Exportación por Defecto (Default Export): Se usa cuando un archivo exporta una sola cosa principal (como una clase o un componente de React):
   - Exportar: Usas export default. Solo puede haber uno por archivo.
   - Importar: No usas llaves. Además, puedes ponerle el nombre que quieras al importarlo.

```
// archivo: Usuario.js
export default class Usuario {
  constructor(nombre) { this.nombre = nombre; }
}


// archivo: app.js
import User from './Usuario.js'; // Lo llamo "User" aunque se exportó como "Usuario"
const alex = new User("Alex");
```

3. Exportación masiva: para importar todo lo que exporte un fichero:

```
import * as Herramientas from './matematicas.js';


console.log(Herramientas.sumar(5, 5));
console.log(Herramientas.PI);
```

Ejercicios (utiliza *document.write('texto'+valor)* para escribir en la página web:

1) Escribe una función mayorQue, que reciba como argumento un número y devuelva una función que represente una prueba lógica. Cuando se llame a esta función devuelta con un único argumento, devolverá verdadero si el número dado es mayor que el número empleado para crear la función de la prueba lógica y falso, en caso contrario.

2) Crea un vector de 10 alturas de persona e informa de cuántos tienen una altura mayor o igual que 1.80m y cuántos menor (usa filter y foreach).

3) Escribe un guión que, dados dos arrays de números enteros, diga cuál tiene el mayor valor acumulado. Escribe una función que haga dicha suma de valores (usa reduce).

4) Crea una función media y completa el ejercicio anterior con la media de cada array

5) Define la clase Vehiculo (marca, modelo, anio_de_matriculacion, anio_de_venta, potencia, cilindrada, combustible), para representar vehículos de segunda mano, crea propiedades de acceso, crea un array de vehículos y escribe funciones para obtener el vehículo más antiguo, determinar cuántos están en un intervalo de años pasados como argumentos y enumerarlos.

1 Bibliografía

- http://www.w3schools.com/js/default.asp
- http://javascript.info
- https://medium.freecodecamp.org/the-complete-javascript-handbook-f26b2c71719c
- https://frontendmasters.com/books/javascript-enlightenment/